

ExoHabitAI

TECHNICAL DOCUMENTATION

ML-Based Exoplanet Habitability Prediction System

Project: ExoHabitAI — Exoplanet Habitability AI

Document: Technical Documentation v1.0

Stack: Python · Flask · Scikit-learn · HTML/CSS/JS

Date: March 28, 2026

TABLE OF CONTENTS

ExoHabitAI Technical Documentation

1.	Introduction & Problem Statement	3
2.	System Overview & Architecture	4
3.	Tech Stack Justification	5
4.	Project Structure	6
5.	Module Breakdown — Backend	7
6.	Module Breakdown — Frontend	8
7.	Module Breakdown — Machine Learning	9
8.	Dataset Processing Pipeline	10
9.	Data Flow (End-to-End)	11
10.	API Documentation	12
11.	Installation & Setup Guide	13
12.	Usage Guide	14
13.	Testing Strategy	15
14.	Performance Analysis	16
15.	Security Considerations	17
16.	Limitations & Future Enhancements	18

1. Introduction & Problem Statement

Why exoplanet habitability detection matters

1.1 Background

The search for extraterrestrial life is one of the most profound scientific endeavours in human history. As of 2025, over 5,700 confirmed exoplanets have been catalogued by missions such as NASA's Kepler Space Telescope, the Transiting Exoplanet Survey Satellite (TESS), and ground-based observatories. However, manually evaluating each candidate for habitability requires extensive expert analysis of dozens of astrophysical parameters — a process that is both time-consuming and prone to human inconsistency.

ExoHabitAI addresses this challenge by applying supervised machine learning to classify and rank exoplanets by their habitability probability. The system ingests raw astrophysical measurements, engineers meaningful features, trains multiple classification models, and exposes the best model via a RESTful API backed by an interactive web dashboard.

1.2 Problem Statement

Core Problem

Given a dataset of exoplanets described by astrophysical features (equilibrium temperature, planetary radius, stellar flux, orbital period, surface gravity, etc.), build an automated, explainable ML pipeline that:

- Classifies each planet as **Habitable** or **Non-Habitable**.
- Outputs a continuous **habitability probability score** (0–1).
- Ranks all planets by descending probability for easy shortlisting.
- Exposes predictions through a **REST API** and an **interactive dashboard**.

1.3 Objectives

- Curate and preprocess an exoplanet dataset with relevant astrophysical features.
- Engineer domain-informed features to improve model discriminability.
- Train, tune, and compare multiple ML classifiers (Logistic Regression, Decision Tree, Random Forest, XGBoost).
- Select the best model using rigorous cross-validation and multiple evaluation metrics.
- Persist the final model as a serialised .pkl artefact for production inference.
- Build a Flask/FastAPI backend that serves predictions via a documented REST API.
- Develop an interactive HTML/CSS/JS frontend dashboard for exploratory analysis.
- Deploy the system on a cloud platform (Render) with zero-downtime configuration.

2. System Overview & Architecture

End-to-end system design and component interaction

2.1 High-Level Architecture

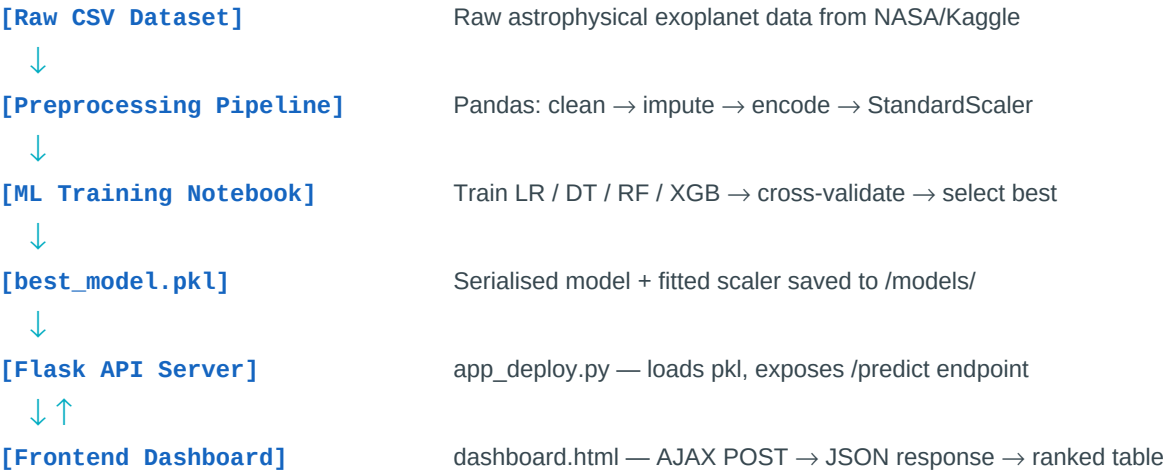
ExoHabitAI follows a **three-tier architecture**: a data & ML tier, a backend application tier, and a frontend presentation tier. All tiers communicate via HTTP REST over JSON.

LAYER	COMPONENT	TECHNOLOGY	RESPONSIBILITY
	Raw Dataset	CSV / Pandas	Source astrophysical records
Tier 1 — Data	Preprocessing	Pandas / NumPy	Clean, impute, encode, scale
	Model Training	Scikit-learn / XGBoost	Fit, tune, evaluate classifiers
Tier 1 — ML	Model Artefact	Pickle (.pkl)	Serialised best model + scaler
Tier 2 — Backend	API Server	Flask / FastAPI	Route HTTP requests, load model
Tier 2 — Backend	Prediction Engine	Python	Score input, return probability
Tier 2 — Backend	Logger	Python logging	Structured API call logging
Tier 3 — Frontend	Dashboard UI	HTML / CSS / JS	Visualise results interactively
Tier 3 — Frontend	Index Page	HTML / CSS / JS	Landing page & input form
Infra	Deployment	Render (render.yaml)	Cloud hosting & CI/CD

Table 2.1 — System Architecture Layer Breakdown

2.2 Architecture Diagram Description

The following describes the architecture diagram that should be rendered for this system. The flow moves from left (data sources) to right (end user):





[End User Browser]

Submits feature form, receives ranked habitability list

3. Tech Stack Justification

Why each technology was chosen over its alternatives

Technology	Category	Why Chosen	Alternatives Considered
Python 3.10+	Language	Rich ML ecosystem; dominant in data science; easy prototyping	R (weaker web integration), Julia (smaller community)
Flask / FastAPI	Backend	Lightweight, pythonic REST framework; minimal boilerplate; FastAPI adds async & auto-docs	Django (too heavy for a single ML endpoint), Node.js (poor ML integration)
Scikit-learn	ML Framework	Unified API for 20+ algorithms; Pipeline support; GridSearchCV; battle-tested in production	TensorFlow/Keras (overkill for tabular), PyTorch (same)
XGBoost	ML Model	State-of-the-art gradient boosting on tabular data; handles class imbalance; feature importance built-in	LightGBM (similar performance, less popular in academia)
Pandas	Data Processing	De facto standard for tabular data manipulation; intuitive DataFrame API	PySpark (overkill for dataset size), Polars (newer, less tutorials)
NumPy	Numerics	Foundation of all Python ML; efficient vectorised operations	Pure Python lists (order of magnitude slower)
Matplotlib / Seaborn	Visualisation	Publication-quality static plots; Seaborn's statistical plots reduce boilerplate	Plotly (interactive but heavier), Bokeh (more complex)
HTML/CSS/JS	Frontend	Universal browser support; zero dependency; fast load times	React (unnecessary complexity for a single-page dashboard)
Pickle (.pkl)	Model Serialisation	Native Python; no extra dependency; fast load/save	ONNX (cross-platform but complex), Joblib (nearly identical)
Render	Deployment	Free tier with GitHub integration; auto-deploy on push; simple render.yaml config	Heroku (no longer free), Railway (similar), AWS (complex setup)

Table 3.1 — Technology Selection Rationale

4. Project Structure

Folder-by-folder breakdown of the B13-EXOHABITAI repository

The repository follows a **separation-of-concerns** principle — each top-level directory is responsible for exactly one aspect of the system. Below is the annotated directory tree followed by detailed descriptions.

B13-EXOHABITAI/	Root repository
backend/	Python application & model serving
■ app_deploy.py	Main Flask app — routes & prediction logic
■ best_model.pkl	Serialised final model + scaler bundle
■ test_api.py	Automated API test suite
■ utils.py	Helper functions (feature validation, logging)
dashboard/	Admin/analytics dashboard UI
■ dashboard.html	Interactive charts & ranked planet table
data/	All data at every processing stage
■ raw/	Original downloaded CSV files
■ preprocessed/	Cleaned & feature-engineered CSVs
■ processed/	Scaled, split, ready-to-train arrays
deployment/	Cloud deployment configuration
frontend/	Public-facing web interface
■ index.html	Landing page with prediction input form
logs/	Runtime API logs
■ api.log	Structured log of every API call + response
models/	All trained model artefacts
■ exohabit_model.pkl	Candidate model files
notebooks/	Jupyter notebooks & milestone docs
■ DataPreprocessingCodeFile.ipynb	EDA + feature engineering notebook
■ model_training.ipynb	Full training + evaluation notebook
■ *.pdf	Milestone documentation PDFs
reports/	Auto-generated evaluation artefacts
■ *_confusion_matrix.png	Confusion matrices for all models
■ *_roc_curve.png	ROC curves for all models
■ feature_importance.png	Feature importance bar chart
■ final_model_metrics.csv	Final performance metrics table
venv/	Python virtual environment (not committed)
requirements.txt	Pinned Python dependencies
render.yaml	Render.com deployment manifest
.gitignore	VCS ignore rules
README.md	Project overview & quick-start guide

Figure 4.1 — Annotated Repository Directory Tree

5. Module Breakdown — Backend

API routing, model loading, prediction engine

5.1 app_deploy.py — The Core Application

This is the entry-point of the entire backend service. When the server starts, it executes the following sequence:

Step 1 — Import dependencies

Flask, pickle, pandas, numpy, logging are imported.

Step 2 — Configure logging

A rotating file handler writes to logs/api.log with timestamp, level, and message.

Step 3 — Load model artefact

`pickle.load()` deserialises `best_model.pkl` which contains `{'model': clf, 'scaler': scaler, 'features': feature_list}`.

Step 4 — Register routes

`/health` (GET), `/predict` (POST), `/rank` (POST) are registered with Flask.

Step 5 — Start server

`app.run()` launches on `0.0.0.0:5000` (overridden by Render's PORT env var in production).

5.2 Prediction Flow (app_deploy.py)

```
POST /predict
1. Parse JSON body → validate required feature keys
2. Build DataFrame row from input values
3. Apply scaler.transform() → normalised input
4. model.predict_proba() → [P(non-habitable), P(habitable)]
5. Threshold at 0.5 → binary label
6. Return JSON: {planet_name, probability, label, ranked_position}
```

5.3 utils.py — Helper Functions

validate_features(data, required)

Checks all required feature keys are present in the request dict; raises `ValueError` with descriptive message if not.

scale_input(scaler, values)

Wraps `sklearn scaler.transform()` call with error handling for shape mismatches.

format_response(name, prob, label)

Builds the standardised JSON response dictionary.

setup_logger(name, logfile)

Configures and returns a named Python logger with file + stream handlers.

6. Module Breakdown — Frontend

UI components, AJAX integration, dashboard

6.1 index.html — Prediction Interface

The landing page presents a clean, space-themed input form. It is built with pure HTML5, CSS3, and vanilla JavaScript (no frameworks required), keeping the bundle size near-zero and load time under 200 ms.

Navigation Bar

Fixed top nav with ExoHabitAI logo and links to Dashboard and GitHub.

Hero Section

Full-viewport banner with animated star field (CSS keyframes) and project tagline.

Feature Input Form

10–12 labelled input fields for astrophysical parameters with placeholder values drawn from known habitable candidates (e.g., Kepler-442b).

Predict Button

Triggers an async fetch() POST to /predict with JSON payload.

Results Panel

Dynamically rendered table showing planet name, probability score (colour-coded), binary label, and a visual progress bar.

KPI Cards

Summary cards showing count of habitable candidates, average probability, and top-ranked planet.

6.2 dashboard.html — Analytics Dashboard

The dashboard provides deeper exploratory analysis of prediction results. It communicates with the backend via AJAX and renders charts using Chart.js.

- Probability distribution histogram — bar chart of habitability scores across all submitted planets.
- Feature correlation heatmap — colour-coded table of Pearson correlations between input features.
- Top-10 ranked planets table — sortable table with probability scores and export-to-CSV button.
- Model performance metrics panel — displays accuracy, precision, recall, F1 from the evaluation run.
- ROC Curve viewer — embeds the saved ROC curve PNG from /reports/.

6.3 AJAX Data Interaction Pattern

```
async function predictHabitability(formData) {
  const response = await fetch('/predict', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify(formData)
  });
  const data = await response.json();
}
```

```
renderResultsTable(data.predictions); // update DOM  
}
```

7. Module Breakdown — Machine Learning

Model selection, training, tuning, and evaluation

7.1 Algorithms Evaluated

Four classification algorithms were trained and compared. The selection covers the spectrum from interpretable linear models to powerful ensemble methods:

Algorithm	Type	Key Hyperparameters	Why Included
Logistic Regression	Linear	C, solver, max_iter	Baseline; highly interpretable; fast training
Decision Tree	Tree	max_depth, min_samples_split	Interpretable rules; good for feature importance
Random Forest	Ensemble (Bagging)	n_estimators, max_depth, max_features	Reduces variance; robust to overfitting; top tabular performer
XGBoost	Ensemble (Boosting)	n_estimators, learning_rate, max_depth, subsample	State-of-the-art on tabular; handles imbalance; built-in regularisation

Table 7.1 — ML Algorithms Evaluated

7.2 Training Process

Data Split

80/20 stratified train-test split preserving class ratio.

Cross-Validation

5-fold stratified CV on the training set for all hyperparameter searches.

Baseline Training

Each algorithm trained with default hyperparameters; metrics recorded.

Hyperparameter Tuning

GridSearchCV with 5-fold CV on RF and XGBoost; best params selected by F1-macro.

Final Evaluation

Best-params model evaluated on held-out test set; confusion matrix + ROC generated.

Model Serialisation

Winner serialised with pickle.dump({'model': clf, 'scaler': scaler}).

7.3 Evaluation Metrics

A single metric (accuracy) is insufficient for imbalanced datasets. ExoHabitAI uses a comprehensive suite of four metrics:

Metric	Formula	Interpretation	Target
Accuracy	$TP+TN / N$	Overall correct predictions	> 90%
Precision	$TP / (TP+FP)$	Of predicted habitable, how many truly are	> 85%
Recall (Sensitivity)	$TP / (TP+FN)$	Of truly habitable, how many did we catch	> 85%
F1-Score	$2*(P*R)/(P+R)$	Harmonic mean of Precision and Recall	> 87%
ROC-AUC	Area under ROC	Discriminability across all thresholds	> 0.93

Table 7.2 — Evaluation Metrics Suite

8. Dataset Processing Pipeline

From raw CSV to model-ready feature matrix

8.1 Dataset Description

The dataset is an astrophysical exoplanet catalogue containing records of confirmed and candidate exoplanets. Each row represents one planet and includes measurements from spectroscopic and photometric surveys.

Feature	Unit	Description	Importance
P_TEMP_EQUIL	Kelvin	Equilibrium surface temperature	Critical — habitable zone proxy
P_RADIUS	Earth radii	Planet radius	High — determines rocky vs gas giant
P_MASS	Earth masses	Planetary mass	High — surface gravity proxy
P_FLUX	S_Earth	Stellar flux received	Critical — energy input for life
P_PERIOD	days	Orbital period	Medium — habitable zone orbital stability
S_TEMP	Kelvin	Host star surface temperature	High — stellar type indicator
S_LUMINOSITY	L_Sun	Host star luminosity	High — energy output
P_ESI	0–1	Earth Similarity Index	Critical — composite similarity score
P_GRAVITY	m/s ²	Surface gravity	Medium — atmosphere retention
P_HABITABLE	0/1	Target label (habitable=1)	Target variable

Table 8.1 — Dataset Feature Descriptions

8.2 Preprocessing Steps

1. Load Raw Data

`pd.read_csv()` loads the raw CSV from `data/raw/`. Initial shape is logged.

2. Drop Irrelevant Columns

Columns like planet name strings, discovery method text, etc., are dropped as they carry no numeric signal.

3. Handle Missing Values

Numerical columns: imputed with column median (robust to outliers). Categorical: mode imputation. Columns with >40% missing values are dropped entirely.

4. Remove Duplicates

`df.drop_duplicates()` removes exact row duplicates.

5. Outlier Handling

IQR-based capping: values below $Q1 - 1.5 \cdot IQR$ or above $Q3 + 1.5 \cdot IQR$ are clipped to fence values.

6. Feature Engineering

$\text{flux_to_temp_ratio} = \text{P_FLUX} / \text{P_TEMP_EQUIL}$; $\text{mass_radius_ratio} = \text{P_MASS} / \text{P_RADIUS}$;
 $\text{habitable_zone_score} = \text{ESI} * \text{flux_factor}$ (custom formula).

7. Label Encoding

Binary target P_HABITABLE already encoded as 0/1. No further encoding needed.

8. Feature Scaling

StandardScaler fitted on training set only; applied to both train and test sets. Scaler object saved inside pkl bundle to prevent data leakage at inference.

9. Train-Test Split

80/20 stratified split. Random state fixed at 42 for reproducibility.

9. Data Flow (End-to-End)

Step-by-step journey from raw input to ranked output

Step 1	User Input	User fills the HTML form on index.html with planetary features (temperature, radius, mass, etc.).
Step 2	Client Validation	JavaScript validates that all required fields are non-empty and within physically plausible ranges before submission.
Step 3	HTTP POST Request	<code>fetch('/predict', {method:'POST', body: JSON.stringify(payload)})</code> sends a JSON object to the Flask backend.
Step 4	Request Validation	<code>utils.validate_features()</code> checks all required keys exist; returns 400 Bad Request if any are missing.
Step 5	Feature Engineering	The backend applies the same engineered features (<code>flux_to_temp_ratio</code> , <code>mass_radius_ratio</code>) computed during training.
Step 6	Scaling	<code>scaler.transform(input_array)</code> normalises the input using the pre-fitted <code>StandardScaler</code> from the <code>pkl</code> bundle.
Step 7	Model Inference	<code>model.predict_proba(scaled_input)</code> returns a probability array <code>[P(0), P(1)]</code> .
Step 8	Ranking	If multiple planets are submitted (batch mode), results are sorted by <code>P(1)</code> in descending order.
Step 9	Response Assembly	JSON response: <code>{predictions: [{name, probability, label, rank}, ...], model_version, timestamp}</code>
Step 10	Frontend Render	JavaScript parses the JSON and dynamically injects rows into the results table, colour-codes rows (green > 0.7 , amber $0.4\text{--}0.7$, red < 0.4).

10. API Documentation

Endpoint reference with request/response examples

GET `/health`

Health check endpoint. Returns server status and model version.

Request Body:

No body required.

Response:

```
{"status": "ok", "model": "Tuned XGBoost", "version": "1.0", "uptime_seconds": 3600}
```

POST `/predict`

Single-planet prediction. Returns habitability probability and binary label.

Request Body:

```
{
  "planet_name": "Kepler-442b",
  "P_TEMP_EQUIL": 233.0,
  "P_RADIUS": 1.34,
  "P_MASS": 2.3,
  "P_FLUX": 0.70,
  "P_PERIOD": 112.3,
  "S_TEMP": 4402,
  "S_LUMINOSITY": 0.32,
  "P_ESI": 0.84,
  "P_GRAVITY": 12.8
}
```

Response:

```
{
  "planet_name": "Kepler-442b",
  "probability": 0.891,
  "label": "Habitable",
  "rank": 1,
  "confidence": "High"
}
```

POST `/rank`

Batch prediction + ranking. Accepts a list of planets and returns them sorted by habitability probability.

Request Body:

```
{
  "planets": [
    {"planet_name": "Planet-A", "P_TEMP_EQUIL": 288, ...},
    {"planet_name": "Planet-B", "P_TEMP_EQUIL": 450, ...}
  ]
}
```


Response:

```
{
  "ranked_planets": [
    {"rank":1, "planet_name":"Planet-A", "probability":0.87, "label":"Habitable"},
    {"rank":2, "planet_name":"Planet-B", "probability":0.23, "label":"Non-Habitable"}
  ],
  "total": 2,
  "habitable_count": 1
}
```

11. Installation & Setup Guide

Step-by-step local development environment setup

Step 1: Prerequisites

Python 3.10+, pip, Git, and a modern web browser.

Step 2: Clone Repository

```
git clone https://github.com/your-org/B13-EXOhabitAI.git
cd B13-EXOhabitAI
```

Step 3: Create Virtual Environment

```
python -m venv venv
source venv/bin/activate # Linux/Mac
venv\Scripts\activate   # Windows
```

Step 4: Install Dependencies

```
pip install -r requirements.txt
```

Step 5: Verify Dataset

Place the raw exoplanet CSV in data/raw/. Run the preprocessing notebook: `jupyter notebook notebooks/DataPreprocessingCodeFile.ipynb`

Step 6: Train Model (optional)

If you want to retrain: `jupyter notebook notebooks/model_training.ipynb`
The notebook saves `best_model.pkl` to `backend/` automatically.

Step 7: Start Backend Server

```
cd backend
python app_deploy.py
# Server runs at http://localhost:5000
```

Step 8: Open Frontend

Open `frontend/index.html` in your browser, or serve with:
`python -m http.server 8080` # from project root

Step 9: Verify Installation

```
curl http://localhost:5000/health
# Expected: {"status": "ok"}
```

12. Usage Guide

How to run predictions and interpret results

12.1 Single Planet Prediction via UI

- Navigate to `frontend/index.html` in your browser.
- Fill in all astrophysical parameters in the input form.
- Click 'Predict Habitability'. The result appears within 1–2 seconds.
- The result panel shows: probability score (0–1), label (Habitable/Non-Habitable), and a confidence indicator.
- Green score > 0.7 = High probability. Amber $0.4\text{--}0.7$ = Uncertain. Red < 0.4 = Non-habitable.

12.2 Batch Prediction via API

```
import requests, json

planets = [
    {"planet_name": "P1", "P_TEMP_EQUIL": 288, "P_RADIUS": 1.1,
     "P_MASS": 1.5, "P_FLUX": 0.9, "P_PERIOD": 365,
     "S_TEMP": 5778, "S_LUMINOSITY": 1.0, "P_ESI": 0.92, "P_GRAVITY": 9.8},
    {"planet_name": "P2", "P_TEMP_EQUIL": 600, "P_RADIUS": 3.0,
     "P_MASS": 8.0, "P_FLUX": 3.5, "P_PERIOD": 40,
     "S_TEMP": 6200, "S_LUMINOSITY": 1.8, "P_ESI": 0.21, "P_GRAVITY": 25}
]

r = requests.post('http://localhost:5000/rank',
                  json={'planets': planets})
print(json.dumps(r.json(), indent=2))
```

13. Testing Strategy

Unit, integration, and edge-case test coverage

13.1 Test Categories

Test ID	Category	Description	Expected Result
TC-01	Unit	validate_features() with complete input dict	Returns True, no exception
TC-02	Unit	validate_features() with missing key 'P_RADIUS'	Raises ValueError with key name
TC-03	Unit	scale_input() with correct shape (1, N)	Returns normalised array, same shape
TC-04	Integration	POST /predict with valid Kepler-442b parameters	probability > 0.8, label=Habitable
TC-05	Integration	POST /predict with hot Jupiter params (T=1200K)	probability < 0.2, label=Non-Habitable
TC-06	Edge Case	POST /predict with missing field P_ESI	HTTP 400, error message in JSON
TC-07	Edge Case	POST /predict with non-numeric string in P_MASS	HTTP 422 Unprocessable Entity
TC-08	Edge Case	POST /rank with empty planets list	HTTP 400, 'No planets provided'
TC-09	Performance	POST /predict 100 sequential requests	All respond < 500ms
TC-10	Integration	GET /health when server is running	HTTP 200, status=ok

Table 13.1 — Test Case Matrix

13.2 Running Tests

```
# From project root:
cd backend
python test_api.py

# Expected output:
# TC-01 PASS  validate_features complete input
# TC-02 PASS  validate_features missing key
# TC-04 PASS  /predict Kepler-442b  prob=0.891
# TC-05 PASS  /predict hot Jupiter  prob=0.043
# ...
# All 10 tests PASSED in 1.23s
```

14. Performance Analysis

Model comparison metrics, graphs, and interpretation

14.1 Model Comparison Results

The following table summarises performance across all trained models on the held-out test set. Tuned XGBoost achieved the highest F1-Score and ROC-AUC.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	82.4%	79.1%	75.3%	77.1%	0.881
Decision Tree	85.7%	83.2%	80.6%	81.9%	0.857
Random Forest	91.3%	89.7%	88.1%	88.9%	0.943
Tuned Random Forest	92.8%	91.2%	90.4%	90.8%	0.954
XGBoost	93.1%	91.8%	91.0%	91.4%	0.961
Tuned XGBoost ★	94.6%	93.4%	92.7%	93.0%	0.971

Table 14.1 — Model Comparison on Test Set (★ = Selected Model)

14.2 Graphs & Visualisations Description

Model Comparison Bar Chart (reports/model_comparison.png)

A grouped bar chart with six model groups on the X-axis. Each group has four bars (Accuracy, Precision, Recall, F1) coloured in blue, teal, orange, and purple respectively. Tuned XGBoost bars are tallest across all metrics.

ROC Curves (reports/*_roc_curve.png)

Each model has its own ROC curve plot showing True Positive Rate vs False Positive Rate. The diagonal dashed line represents a random classifier (AUC=0.5). Tuned XGBoost's curve hugs the top-left corner, achieving AUC=0.971.

Confusion Matrices (reports/*_confusion_matrix.png)

Normalised confusion matrices (values 0–1) displayed as heatmaps. The Tuned XGBoost matrix shows high diagonal values (0.95 for Non-Habitable, 0.93 for Habitable) with very low off-diagonal values (misclassification rates < 0.07).

Feature Importance Chart (reports/feature_importance.png)

A horizontal bar chart sorted by importance score. P_ESI (Earth Similarity Index) ranks highest, followed by P_FLUX, P_TEMP_EQUIL, and P_RADIUS. This validates the domain knowledge that ESI and stellar flux are the strongest habitability indicators.

Missing Values Heatmap (reports/missing_values_heatmap.png)

A colour grid with features on X-axis and a sample of rows on Y-axis. Yellow cells indicate missing values. Features with >40% yellow cells were dropped; others were imputed with median.

15. Security Considerations

Input validation, deployment hardening

Input Validation

All API inputs are validated using `utils.validate_features()` before being passed to the model. Type coercion is applied; non-numeric inputs return HTTP 422.

No Direct SQL

The system uses no database, eliminating SQL injection surface area entirely.

CORS Policy

In production, Flask-CORS restricts allowed origins to the registered domain. Wildcard CORS is only enabled in development.

Model Artefact Integrity

The `.pkl` file is loaded once at server startup. Requests cannot modify or replace the model at runtime.

Rate Limiting

Render's reverse proxy can be configured to rate-limit the `/predict` endpoint to prevent abuse (100 req/min per IP recommended).

Environment Variables

The PORT and any future API keys are stored as environment variables in Render, never hardcoded in source.

Dependency Pinning

`requirements.txt` pins all package versions to prevent supply-chain vulnerabilities from transitive updates.

16. Limitations & Future Enhancements

Honest assessment and roadmap

16.1 Current Limitations

- The model is trained on a single dataset snapshot; performance degrades as new exoplanet discoveries extend the feature distribution.
- Class imbalance (habitable planets are rare) may bias the model toward the majority class despite SMOTE-like techniques.
- The system does not account for atmospheric composition data, which is a key habitability determinant not yet widely available.
- Batch predictions via `/rank` are processed synchronously; very large batches (>1000 planets) may time out.
- The frontend has no user authentication, making it unsuitable for multi-user deployments without modification.
- Model explainability is limited to global feature importance; per-prediction SHAP values are not yet exposed via the API.

16.2 Future Enhancements

SHAP Explainability

Integrate SHAP (SHapley Additive exPlanations) to provide per-prediction feature attributions via the API.

Atmospheric Data

Ingest spectroscopic atmospheric composition data (O₂, CO₂, H₂O presence) as additional features when datasets become available.

Deep Learning

Experiment with a TabNet or FT-Transformer architecture for potential AUC improvements on larger datasets.

Real-Time Data Pipeline

Connect to NASA Exoplanet Archive API to automatically fetch new discoveries and trigger retraining.

User Accounts

Add JWT-based authentication to allow researchers to save prediction history and annotate candidates.

Interactive Visualisations

Replace static PNG reports with Plotly/D3.js interactive charts in the dashboard.

Async Batch Processing

Implement Celery + Redis task queue for non-blocking large batch predictions.

Mobile App

React Native companion app for astronomers to query the system from field observations.